

Matrix

```
def gauss_solver_matrix(A, b, x_0,
                        accuracy: int = 1e-4):
```

- Genauigkeit wird als default übergeben
- L, R, D mithilfe von np.tril, np.triu, np.diag

while-Schleife

```
# Inverse berechnen
inverse = np.linalg.inv(D + L)

while r >= accuracy:
    # Gauss-Seidel-Verfahren
    x_m_new = -inverse @ R @ x_m + inverse @ b
    # Berechnung des Residuums
    r = np.linalg.norm(b - A @ x_m_new)
    # Alten Iterationschritt ueberschreiben
    x_m = x_m_new
```

- Abbruchkriterium entspricht Betrag des Residuums

Komponenten

- Ein Summenausdruck kann als Skalarprodukt interpretiert werden
- Da die Vektoren immer zusammenhängen kann mit Slicing gearbeitet werden

$$\begin{array}{c}
 \boxed{i=2 \quad x_{1,2} \text{ berechnen}} \\
 \begin{pmatrix} 10 & 2 & -1 & 1 \\ 1 & 8 & 2 & -2 \\ -1 & 3 & 12 & 4 \\ 2 & -1 & 3 & 15 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 13 \\ 9 \\ 18 \\ 19 \end{pmatrix} \quad x_0 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} \quad x_{n+1} = \begin{pmatrix} 2,3 \\ 0 \\ 0 \\ 0 \end{pmatrix}
 \end{array}$$

$$x_{1,2} = \frac{1}{a_{22}} (b_2 - a_{21}x_{11} - a_{23}x_{13} - a_{24}x_{14})$$

```
while r >= accuracy:
    for i in range(n): # Zeilenindex i
        # Summen entsprechen Skalarprodukten
        x[i] = ((b[i] -
                np.dot(A[i, :], x[:i]) - #
                    Summe der aktuellen
                    Iteration
                np.dot(A[i, i+1:], x[i+1:])) #
                    Summe der letzten
                    Iteration
                / A[i, i])
```

- $:i = i - 1 \rightarrow i$ exklusiv
- Anstelle von np.dot hätte auch @ verwendet werden können

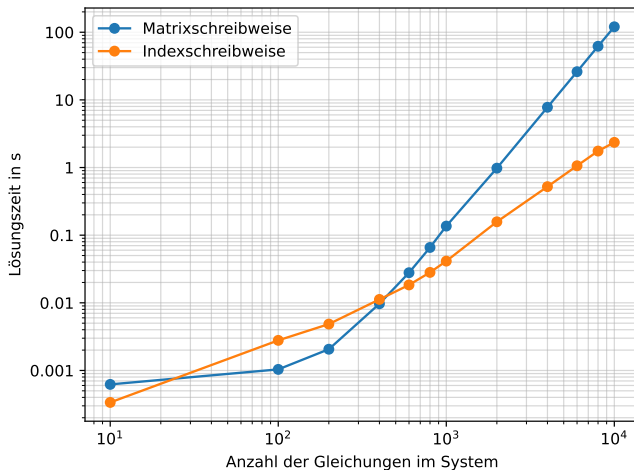


Abbildung: Laufzeitenvergleich